# DevOps Explanation Request

Alright vicky 😈

**DAY-31: CoreDNS in Kubernetes** — the silent hero.

When it breaks, *everything* looks broken.

Same flow as always 👇

👉 **way → problem → how Kubernetes solves it → hands-on → deep internals → traps**

---

## ☠️ Way (Without DNS in a cluster)

Imagine this life:

- Pods talk to each other using **IP addresses**
- IPs change when:
  - Pod restarts
  - Pod reschedules
  - Node dies
- Every config update needs IP change

Result:

> Distributed system turns into distributed headache 💀

---

## 💣 Problem

Kubernetes is **dynamic by design**:

- Pods are ephemeral
- Services get virtual IPs
- Scaling changes endpoints constantly

So apps need:

- **Stable names**
- **Dynamic IP resolution**
- **Automatic updates**

Hardcoding IPs is impossible.

---

## 🛠️ How Kubernetes Solves This

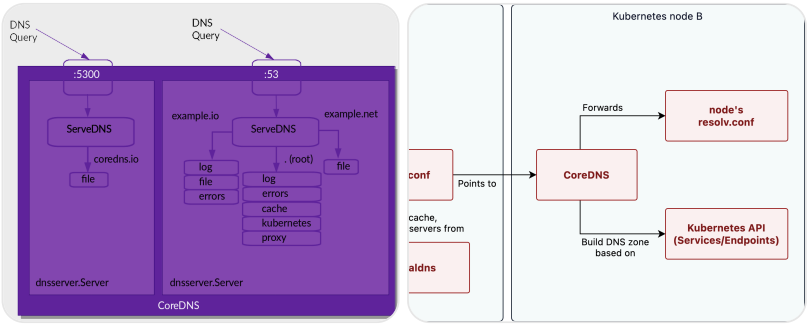
Using **CoreDNS**

> **CoreDNS provides DNS-based service discovery inside the cluster**

It maps:

```pgsql
Service Name  → ClusterIP
Pod Name      → Pod IP (optional)
```

Kubernetes installs CoreDNS by default.




---

# Step 1 – What CoreDNS actually does

CoreDNS answers DNS queries like:

```pgsql
my-service.default.svc.cluster.local
```

And returns:

```nginx
ClusterIP of the service
```

Then kube-proxy takes over and forwards traffic to pods.

---

## DNS naming format (VERY IMPORTANT)

```pgsql
```

```
<service-name>.<namespace>.svc.cluster.local
```

Examples:

- `redis.default.svc.cluster.local`
- `api.dev.svc.cluster.local`

Inside same namespace:

```bash
curl http://redis
```

Works because DNS search paths are injected.

---

## Step 2 – Where CoreDNS runs

CoreDNS runs as **pods in kube-system namespace**.

Check:

```bash
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Example output:

```sql
coredns-7b96bf9f76-5hsxb   Running
coredns-7b96bf9f76-mvmmt   Running
```

Why multiple pods?
👉 **High availability**

---

### What if CoreDNS pods are down?

Symptoms:

- `curl service-name` fails
- Inter-service communication fails

- Everything looks “network broken”

Reality:

> Network is fine. DNS is dead.

---

## Step 3 – How DNS works inside a Pod

Every pod gets a file:

```bash
/etc/resolv.conf
```

Example:

```text
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
```

- `10.96.0.10` → CoreDNS **Service IP**
- All DNS queries go here

So flow becomes:

```nginx
Pod → CoreDNS → Kubernetes API → Answer
```

---

## Step 4 – CoreDNS internal working (simple)

When CoreDNS starts:

1. It watches:
   - Services
   - Endpoints
2. Builds DNS records dynamically
3. Updates records **in real time**

No restart needed when services scale.

---

## Step 5 – Hands-on DNS testing (must know)

### Create a test pod

```bash
kubectl run dns-test --image=busybox:1.28 -it --rm -- sh
```

Inside pod:

```bash
nslookup kubernetes.default
```

Result:

```vbnet
Name: kubernetes.default
Address: <ClusterIP>
```

If this fails → CoreDNS problem.

---

### Test service resolution

```bash
nslookup my-service.default.svc.cluster.local
```

or simply:

```bash
ping my-service
```

---

## Step 6 – CoreDNS ConfigMap (advanced but important)

CoreDNS config lives in:

```bash
kubectl get configmap coredns -n kube-system -o yaml
```

Key file: **Corefile**

This controls:

- Kubernetes plugin
- Forwarding to external DNS
- Caching

⚠️ Breaking this file = cluster-wide outage

---

## Step 7 – Networking + CoreDNS (CALICO case)

Now the **Calico part you mentioned** — this is VERY real.

### Why CoreDNS fails on multi-node cloud setups

On cloud (AWS, etc):

- Nodes are VMs
- Cloud has **Source/Destination Check enabled**
- Calico uses **overlay networking**

If Source/Dest check is enabled:
× Packets get dropped
× Pod-to-pod traffic fails
× CoreDNS unreachable

---

### Fix for AWS + Calico

#### Disable Source/Destination Check

On **each EC2 worker node**:

- Disable source/destination check

This allows:

- Pod IPs to route correctly

---

### Calico specifics

- Calico runs as **DaemonSet**
- One Calico pod per node
- Responsible for:
  - Pod networking
  - Routing
  - Network policy

If Calico is down:

- CoreDNS may be up
- But unreachable → DNS fails anyway

---

### IP_AUTODETECTION (Important)

Calico needs to detect **correct node IP**.

If wrong IP is detected:

- Pod network breaks
- DNS resolution fails indirectly

Setting:

```nginx
IP_AUTODETECTION_METHOD
```

fixes multi-NIC / cloud issues.

---

## Step 8 – Common CoreDNS Failure Symptoms

| Symptom | Root cause |
|---|---|
| Services unreachable | CoreDNS down |
| External DNS fails | CoreDNS forward issue |
| Works on same pod | DNS search path issue |

| Symptom | Root cause |
| --- | --- |
| Works with IP only | DNS resolution broken |
| Multi-node issues | CNI / Calico misconfig |

---

## Step 9 – Common Mistakes (Seen in prod)

× Assuming DNS = networking
× Restarting apps instead of checking CoreDNS
× Breaking Corefile
× Forgetting cloud networking rules
× Ignoring CNI health

---

## 🧠 Final Mental Model (Lock this)

```nginx
Service → DNS name → CoreDNS → ClusterIP → kube-proxy → Pod
```

Or simpler:

> **If services can't talk by name, CoreDNS is guilty until proven innocent.**

---

If you want next 🔥

- CoreDNS vs kube-dns
- Debugging DNS step-by-step (real checklist)
- ExternalName services
- Headless services & DNS
- How StatefulSet uses DNS

Say the word 😉